

Designed by Remus Teodorescu 2026

LECTURE 4.2

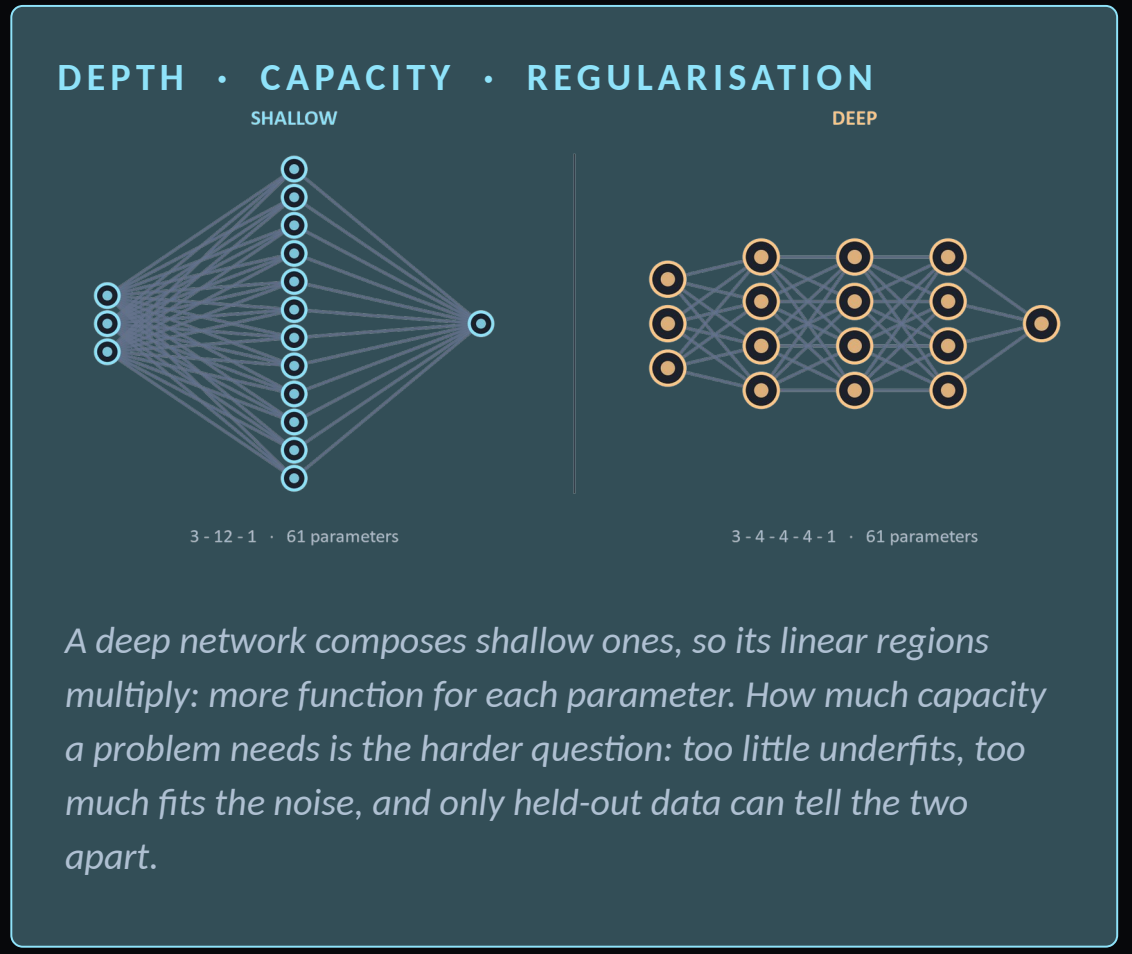
Deep Neural Networks

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026



Learning Objectives

SECTIONS, IN ORDER

1. **Deep networks** — composition, layer notation, parameter count, what depth buys
2. **Capacity** — underfitting and overfitting, data splits, bias and variance
3. **Regularisation** — weight decay, early stopping, and choosing capacity

As in the last lecture, every curve is already fitted: the optimiser stays a black box until lecture six.

READ, AND WHAT EACH ANSWERS

UDL ch. 4	what does depth add to a network?
UDL ch. 8	how much capacity does a problem need?
UDL ch. 9	how is a flexible model kept honest?

Composition

Deep network — a network with more than one hidden layer: the output of one layer is the input of the next, so it is shallow networks composed. Each layer re-describes its input, and the next layer works in that new description; the neurons, activations and loss are the same as before.

ONE NETWORK FED INTO THE NEXT

- **each layer** — a representation the next one works in
- **K layers** — K compositions of the one-layer function

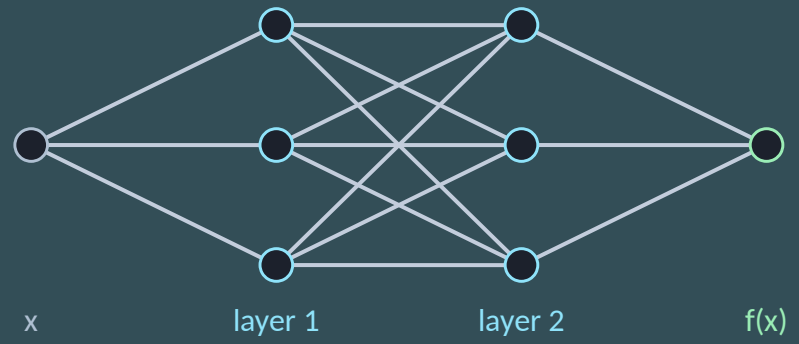
$$f(x) = f_K \circ f_{K-1} \circ \cdots \circ f_1(x)$$

f_k the kth layer, K how many layers, $\circ$ means feed the output onward

COMPOSITION IN PYTORCH

```
deep = nn.Sequential(nn.Linear(1, 3), nn.ReLU(),  # f₁: the first layer
                     nn.Linear(3, 3), nn.ReLU(),  # f₂: works in f₁'s output
                     nn.Linear(3, 1))             # the output: f₃
```

TWO SHALLOW NETWORKS, JOINED



WHAT A LAYER DOES

A layer transforms the space; the next layer solves an easier problem in the transformed space. The hidden layer that solved XOR did this once. Depth does it K times.

Layer Notation

Layer notation — one layer written with a weight matrix $\mathbf{W}$ and a bias vector $\mathbf{b}$: it computes $\mathbf{h} = a(\mathbf{W}\mathbf{x} + \mathbf{b})$, the activation applied to each element. $\mathbf{W}$ has one row per neuron and one column per input, so its shape follows from the sizes of the two layers it joins.

WEIGHTS AS A MATRIX, BIASES AS A VECTOR

- **the output of layer k** — is the input of layer k + 1
- **in PyTorch** — nn.Linear holds W and b

$$\mathbf{h}_k = a(\mathbf{W}_k \mathbf{h}_{k-1} + \mathbf{b}_k)$$

$\mathbf{h}_k$ the output of layer k, $\mathbf{W}_k$ its weight matrix, $\mathbf{b}_k$ its bias vector, a the activation

$$\underset{64\times 1}{\mathbf{h}_1} = a\Big(\underset{64\times 3}{\mathbf{W}_1}\ \underset{3\times 1}{\mathbf{x}} + \underset{64\times 1}{\mathbf{b}_1}\Big)$$

ONE LAYER IN PYTORCH

```
layer = nn.Linear(3, 64)  # W₁ and b₁: 256 parameters
layer.weight.shape        # (64, 3): 192 numbers
layer.bias.shape          # (64,): 64 numbers
h1 = torch.relu(layer(x)) # h₁ = a(W₁x + b₁)
```

PREDICT

A second layer of 64 neurons follows. What shapes are W_2 and b_2, and how many numbers? The next slide counts them.

Parameter Count

Parameter count — the number of weights and biases a network learns. A layer joining D_1 neurons to D_2 neurons holds $D_1 \cdot D_2$ weights and D_2 biases, so the count grows with the square of the width and in proportion to the depth.

SQUARE IN THE WIDTH, LINEAR IN THE DEPTH

- **double the width** — about four times the parameters
- **each extra layer** — adds $D^2 + D$, whatever the depth

$$P = D_i D + (K-1)D^2 + DD_o + (KD + D_o)$$

P parameters, D_i inputs, D the width, K hidden layers, D_o outputs

$K = 1$ $\quad P = \underset{\text{input}}{3 \cdot 64} + \underset{\text{hidden}}{0} + \underset{\text{output}}{64 \cdot 1} + \underset{\text{biases}}{(64+1)} = 321$

$K = 2$ $\quad P = \underset{\text{input}}{3 \cdot 64} + \underset{\text{hidden}}{64^2} + \underset{\text{output}}{64 \cdot 1} + \underset{\text{biases}}{(2 \cdot 64 + 1)} = 4\,481$

PREDICT

Width 128, two hidden layers: four times 4,481, more, or less?

SOME REAL COUNTS

1 × 10 × 1	31 — one hidden layer of ten
2 × 32 × 32 × 1	1,185 = 64 + 1,024 + 32 + 65
4 layers of 50	about 8,000 — a flow field
ResNet-50	25 million
a 2020 language model	175 billion

COUNT THEM IN PYTORCH

```
net = nn.Sequential(nn.Linear(3, 64), nn.ReLU(),  # 256
                    nn.Linear(64, 64), nn.ReLU(), # 4,160
                    nn.Linear(64, 1))             # 65
sum(p.numel() for p in net.parameters())         # 4,481
```

Depth Efficiency

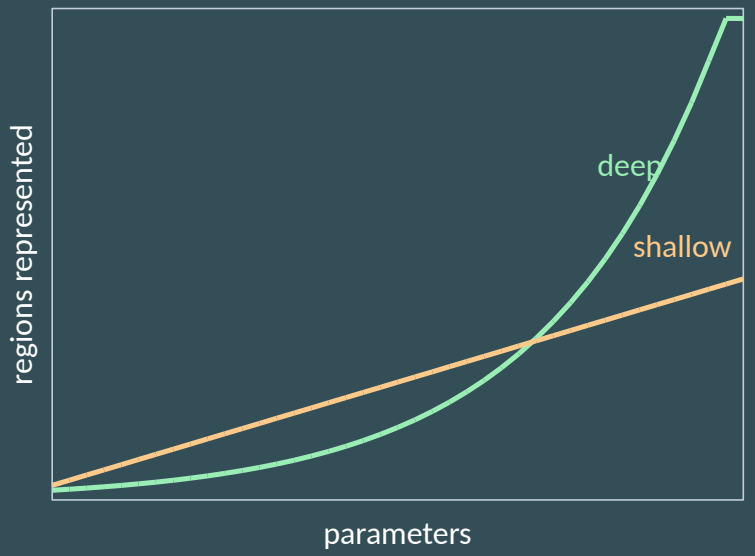
Depth efficiency — a deep network can make many more linear regions than a shallow network with the same number of parameters, because each layer folds the output of the one before and the regions multiply. Some functions need exponentially more neurons if the network is shallow.

REGIONS MULTIPLY WITH DEPTH

- **one layer of D neurons** — D + 1 linear regions
- **K layers of D neurons** — roughly D to the power K
- **in practice** — start at 4 × 32; change it on evidence

TALK · 2 MIN How would you explain what depth adds to a network, in your own words? Then, in the plot on the right, why does the deep curve pull away from the shallow one as the parameters grow?

REGIONS AGAINST PARAMETERS



Schematic: no units, the claim is the ordering. Notebook 04 measures it at a fixed parameter budget.

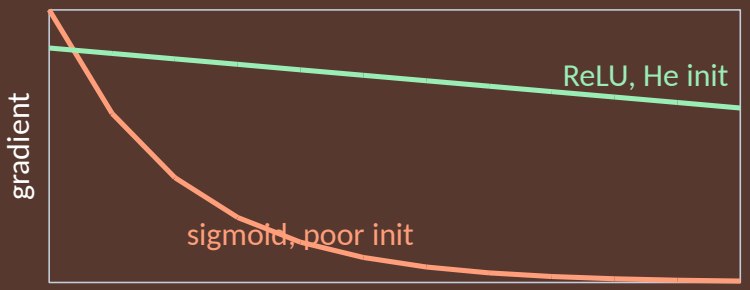
Training Difficulties

Training difficulties — the gradient that reaches the early layers of a deep network is a product of one factor per layer; with factors below one it vanishes, and those layers barely learn. ReLU activations, careful initialisation and, for very deep networks, residual connections made depth trainable.

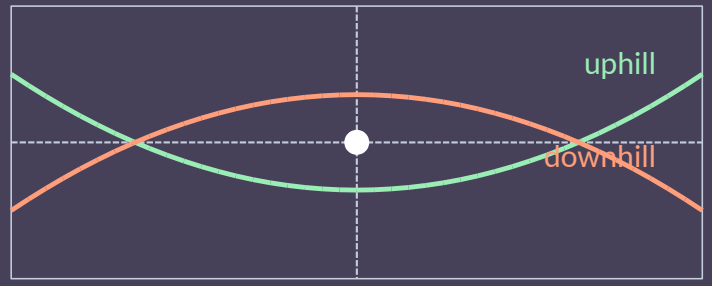
TWO REASONS DEPTH WAS HARD

- **vanishing gradient** — one factor per layer, below one
- **the fix** — ReLU, He initialisation, residual links
- **many minima** — mostly saddles, which descent escapes

GRADIENT SIZE BY LAYER



A SADDLE, NOT A BOWL



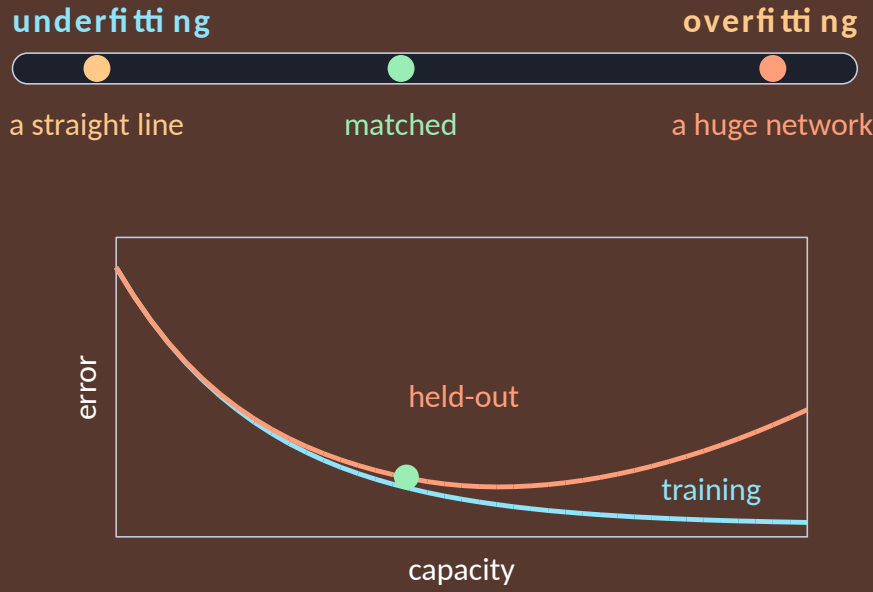
Model Capacity

Model capacity — the range of functions a model can represent, set mostly by its number of parameters. Too little and the model cannot follow the function behind the data; too much and it follows the noise as well. The training error keeps falling as capacity grows; the error on new data does not.

HOW MANY FUNCTIONS A MODEL CAN REPRESENT

- **the aim** — the function behind the data, not every function
- **training error** — falls with capacity, without limit
- **held-out error** — falls, then rises again

ERROR AGAINST CAPACITY



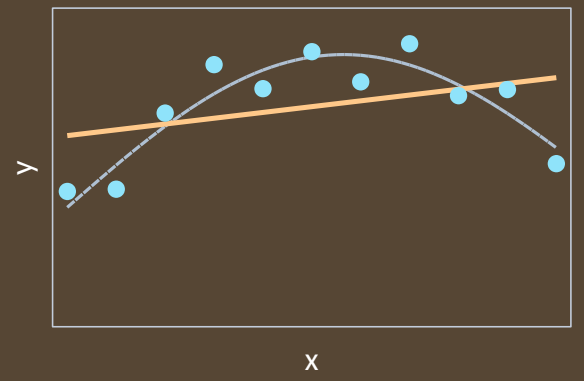
The green point is the capacity to aim for. It is found only with data held out of training.

Underfitting and Overfitting

Underfitting and overfitting — underfitting is a model too simple to follow the data: the training error stays high. Overfitting is a model that follows the noise in its training data: the training error is low, but the error on new data is high. Only data held out of training tells either from a good fit.

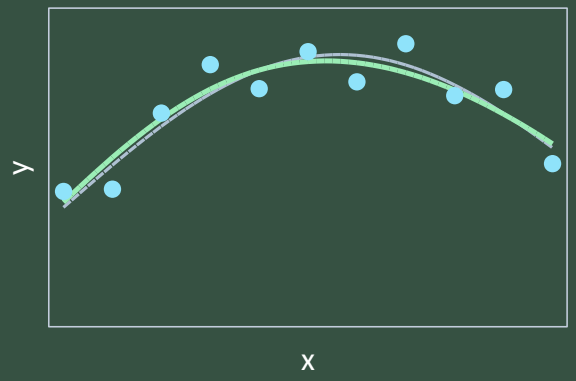
TALK · 2 MIN How would you explain overfitting to a colleague? Then, since you never have the dashed curve, what measurement would show that the third fit is the wrong one?

UNDERFITTING



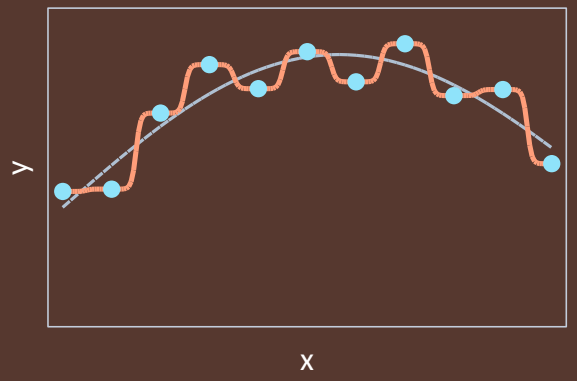
training error high, held-out high

MATCHED CAPACITY



training moderate, held-out lowest

OVERFITTING



training error ≈ 0, held-out worst

Data Splits

Data splits — the data divided into three sets with three jobs: the training set fits the parameters, the validation set chooses between models and settings, and the test set, used once at the end, estimates the error on new data. Samples that are not independent stay in the same set.

TRAINING, VALIDATION AND TEST

- **training set** — fits the parameters
- **validation set** — chooses the model
- **test set** — used once, at the end



HOW TO SPLIT

independent	at random
time series	by time, never at random
repeated runs	by run, not by sample
one rig, one batch	less data than it looks
small N	cross-validate

Two samples from the same test run are not independent. Split them at random and the training set contains the answer: the score is excellent and the model has learned nothing.

Bias and Variance

Bias and variance — two parts of a model's error besides the noise. Bias is how far the average fitted model, over many possible training sets, lies from the truth; variance is how much the fitted model changes from one training set to another. More capacity lowers the bias and raises the variance.

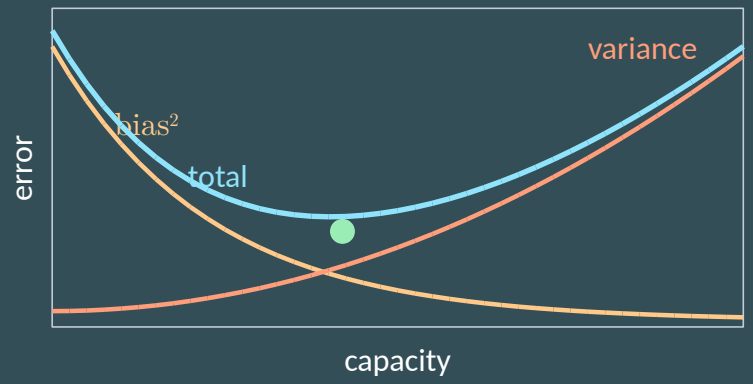
THREE PARTS OF THE ERROR

- **noise** — the floor no model removes
- **repeat the experiment** — new data, refit each time
- **below the noise floor** — suspect leakage first

$$\mathbb{E}\left[\left(y - \hat{f}\right)^2\right] = \text{bias}^2 + \text{variance} + \sigma^2$$

y the truth, $\hat{f}$ the fitted model, $\mathbb{E}$ averaged over refits, σ^2 the noise floor

BIAS AND VARIANCE AGAINST CAPACITY



BIAS AND VARIANCE, MEASURED

```
preds = torch.stack([fit(s)(x) for s in range(20)])  # 20 fits
bias2 = (preds.mean(0) - truth(x)).pow(2).mean()  # bias²
var = preds.var(0).mean()                         # variance
```

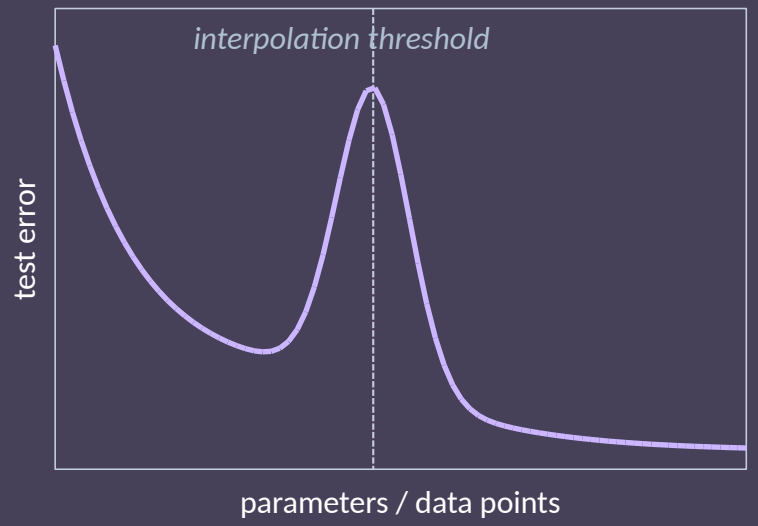
Double Descent

Double descent — the observation that the test error, after rising as the model becomes just large enough to fit the training data exactly, can fall again as the model grows far beyond that point. Among the many exact fits, training tends to find a smooth one.

PAST THE INTERPOLATION THRESHOLD

- **the U** — fewer parameters than data points
- **the peak** — where the model can just fit the data
- **in practice** — a few hundred samples: left of the peak

TEST ERROR AGAINST CAPACITY



Schematic. The dashed line is where the model can just fit the training set exactly: left of it, the classical U; right of it, the second descent.

Regularisation

Regularisation — any change to training that lowers the error on new data rather than on the training data. The most common is weight decay, which adds the sum of the squared parameters to the loss so that large weights cost more; early stopping, dropout and data augmentation are others.

WEIGHT DECAY

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \lambda \sum_j \theta_j^2$$

$\mathcal{L}$ the data loss, $\tilde{\mathcal{L}}$ the regularised loss, θ_j each parameter, λ how much you care

WEIGHT DECAY IN PYTORCH

```
l2 = sum(p.pow(2).sum() for p in model.parameters())  # Σθ²
loss = loss_fn(model(x), y) + lam * l2                # the regularised loss
opt = torch.optim.SGD(params, lr=0.01, weight_decay=lam)  # built in
```

- **the others** — early stopping, dropout, a smaller model

WHERE EACH IS USED

weight decay	Ex_04 notebook 05
early stopping	every training run
dropout	large classifiers; rarely here
augmentation	camera images, L11
a smaller model	the usual answer here
physics in the loss	Part 2 — the strongest of all

A physics residual is a regulariser: it penalises every violation of a conservation law, everywhere it is evaluated. Weight decay only discourages large weights.

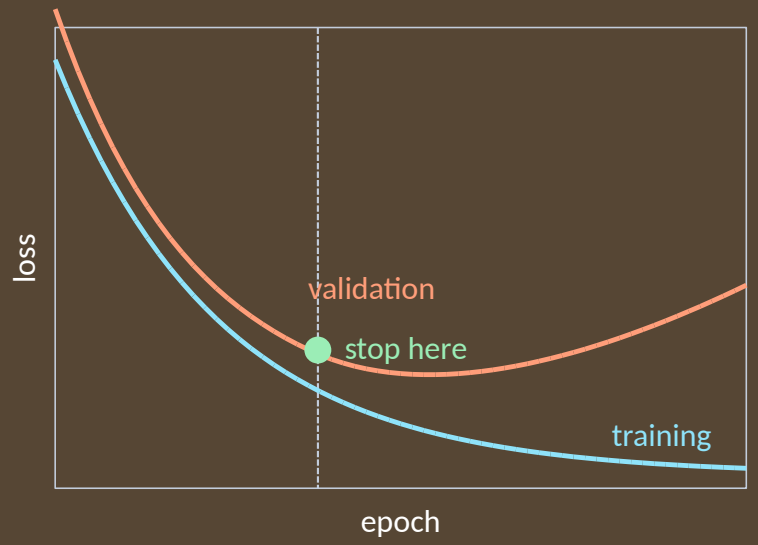
Early Stopping

Early stopping — ending training when the validation loss stops falling, and keeping the parameters from that point. The training loss keeps falling as the model begins to fit the noise; the validation loss turns up, and the turn is where to stop.

TRAINING AND VALIDATION LOSS, ON ONE PLOT

- **plot both losses** — on one set of axes, every run
- **validation turns up** — the model is fitting noise
- **keep** — the lowest-validation checkpoint
- **flat and high** — underfit; a widening gap — overfit

TRAINING AND VALIDATION LOSS



Right of the dashed line the model is memorising the training set. Keep the checkpoint from the green point.

Choosing Capacity

Choosing capacity — finding the network size and regularisation that give the lowest validation error. There is no formula: start small, grow the network until it fits the training data, then regularise, and change the architecture only when the validation error says so.

A PROCEDURE, NOT A FORMULA

- **start too small** — the underfit gives a floor
- **regularise before shrinking** — decay, early stopping
- **search at random** — a grid wastes trials

WITHOUT LABELLED DATA

A network trained on a physical law has no samples to hold out. Hold out conditions instead, check the residual at points not trained on, and check the conservation laws.

THE ORDER TO WORK IN

1	overfit five samples: is it wired up?
2	start small; note the underfit floor
3	grow until the training loss is low
4	plot training and validation together
5	add weight decay and early stopping
6	only then change the architecture

Step one belongs at the top of every procedure: a model that cannot memorise five points has a bug, and no architecture search fixes a bug.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
residual links	trains a hundred layers	UDL ch. 11.2	very deep vision models
normalisation	rescales each layer's input	UDL ch. 11.4	deep nets, large batches
dropout	switches off random neurons	UDL ch. 9.3.3	large classifiers
augmentation	transformed copies of the data	UDL ch. 9.3.8	images, camera data
ensembles	averages several models	UDL ch. 9.3.2	uncertainty estimates
Bayesian nets	a distribution over weights	UDL ch. 9.3.5	risk-aware decisions

WHY THEY ARE LEFT OUT

They matter for very deep networks and very large data sets. Weight decay, early stopping and a right-sized network cover the problems in this course.

Exercise 4

THE NOTEBOOKS, IN ORDER

- **00 to 03** — L4.1's: the setup, XOR, PyTorch, linear regions
- **04** — depth against width: one parameter budget, spent three ways
- **05** — overfit the eleven points of this lecture, then regularise
- **06** — the report: every notebook's answers, and one question to conclude

THE QUESTION EACH NOTEBOOK ANSWERS

04	deeper or wider, at equal budget?
05	what does regularisation recover?

REPORT QUESTION, TO CONCLUDE

Across the notebooks: what decided how well your network fitted — the activation, the width, the depth or the data — and what number shows it?

SUMMARY

Key Takeaways

DEEP NETWORK

what it is	shallow networks composed, layer after layer
function	linear regions that multiply with depth
train, tune	ReLU, He initialisation; start at 4 × 32
vs shallow	more regions for the same parameters
used for	every network from here on, Part 2's too

GENERALISATION

what it is	a low error on data the model has not seen
function	set by capacity: too little, matched, too much
train, tune	validation set; weight decay, early stopping
vs training	an overfit: training ≈ 0, held-out worst
used for	choosing, and trusting, every model you ship

Next — L5.1: convolutional and graph networks.

Questions

1 – 2 · DEEP NETWORK

1 **What does a deep network gain over a wide one with the same number of weights?**

Each layer folds the output of the layer before, so the number of straight pieces multiplies with depth, while width only adds to it. For the same number of weights a deep network can describe a more detailed function. *Go deeper: deep against wide (Ex_04 notebook 04); UDL ch. 4.*

2 **Why were deep networks hard to train, and how large should a first one be?**

The gradient reaches the first layers as a product of many factors and can shrink to almost nothing on the way. ReLU and He initialisation keep it alive. Start with four layers of 32 neurons, and change the size only when the error on unseen data says so. *Go deeper: the gradient at each layer (Ex_04 notebook 04); UDL ch. 7.5 and 11.*

3 – 4 · GENERALISATION

3 **How do the training error and the error on unseen data change as a network grows?**

The training error keeps falling. The error on unseen data falls, then rises once the network starts to fit the noise. A curve through every training point with a poor error on unseen data is an overfit. *Go deeper: both errors against the size of the network (Ex_04 notebook 05); UDL ch. 8.1 – 8.2.*

4 **How is the size of a network chosen, and how is it kept from overfitting?**

With three sets: train on the first, choose the size and the settings on the second, and report the third once, at the end. Before shrinking the network, try weight decay and early stopping. *Go deeper: early stopping and weight decay (Ex_04 notebook 05); UDL ch. 8.5 and 9.1.*

NEXT

L5.1 — Convolutional and Graph Networks

So far every layer has connected everything to everything. Next: one learned local rule applied everywhere, on a grid and then on a graph.

BEFORE THE NEXT SESSION

read	UDL ch. 10.1 – 10.2
run	Ex_04 notebooks 04 and 05
bring	the two loss curves from notebook 05
think about	why a dense layer on an image is wasteful

Bring the curve. L5.1 opens by asking three people to describe the shape of theirs.